이화여자대학교

Final Project Presentation

Tunnel Inspection Robot for Crack Detection and Risk Mapping

제로 JERO
2391019 이정연
2391022 정선영

List of Contents

01. Project Goal

02. System Architecture

03. Implementation Method

04. Result & Demo

05. Troubleshooting & Discussion

Project Goal

01. Project Goal

02. System Architecture

03. Implementation Method

04. Result & Demo

05. Troubleshooting & Discussion

Project Goal

Tunnel Inspection Robot for Crack Detection and Risk Mapping

자율 주행 로봇이 터널 중앙을 주행하며 RGB-D 카메라로 주변 벽면의 균열을 실시간으로 인식하고, 실제 균열 크기를 산출하여 위험도를 판별한 뒤, 균열 위치와 위험 등급을 지도 위에 시각화하는 터널 안전 진단 시스템





Motivation

Why a Tunnel Inspection Robot?

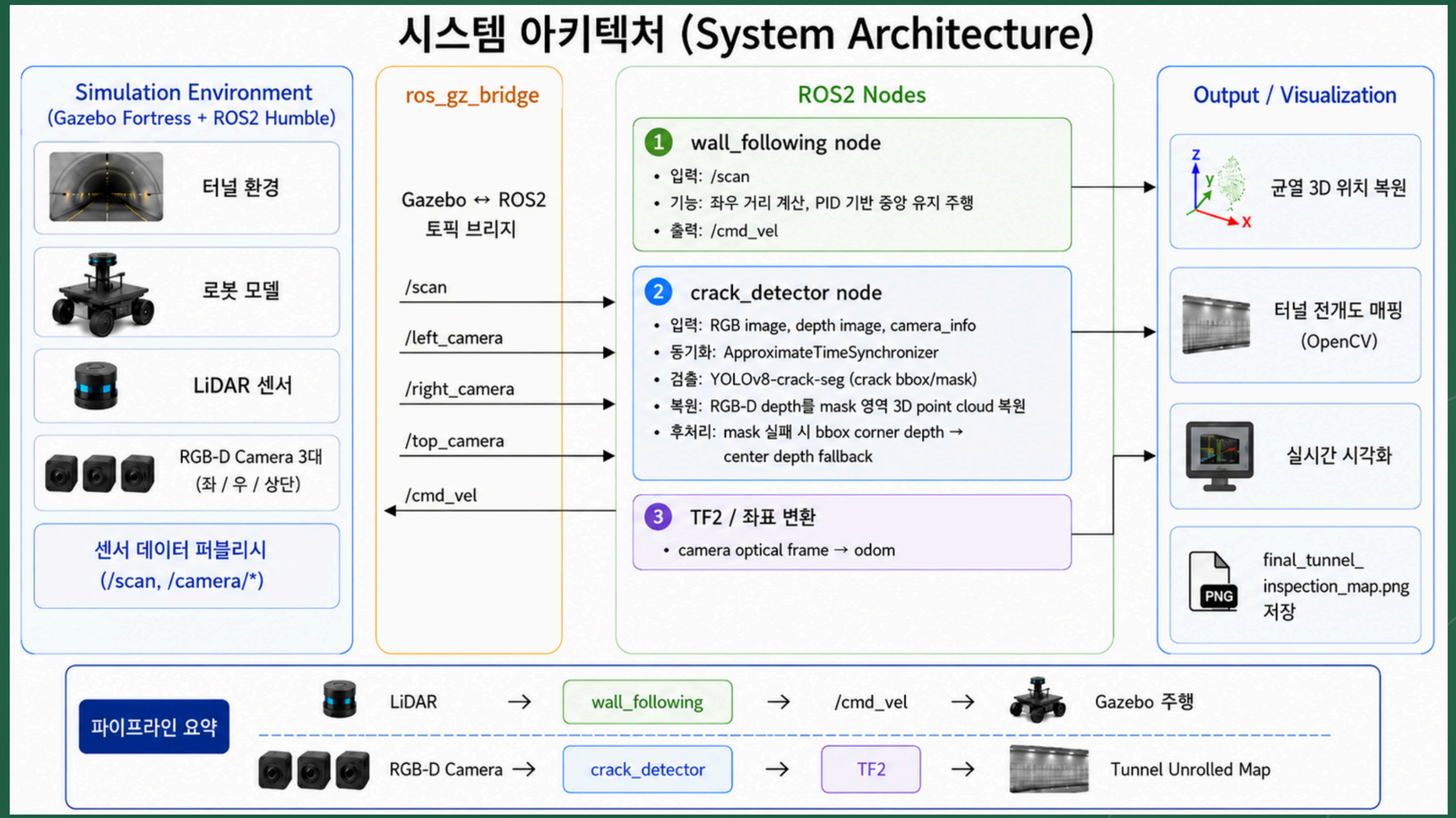
- 터널은 장기간 사용될수록 균열, 누수, 표면 손상 등 구조적 결함이 발생할 수 있음
- 기존 터널 점검은 **사람이 직접 이동**하며 **육안으로 확인**하는 방식에 의존하는 경우가 많음
- 반복적이고 위험할 수 있는 점검 과정을 **로봇으로 자동화**하면 **안전성**과 **효율성**을 높일 수 있음

📌 본 프로젝트는 **균열을 찾는 것**에서 끝나지 않고 **어디에 있는지 지도화**하는 데 초점

System Architecture

01. Project Goal
02. System Architecture
03. Implementation Method
04. Result & Demo
05. Troubleshooting & Discussion

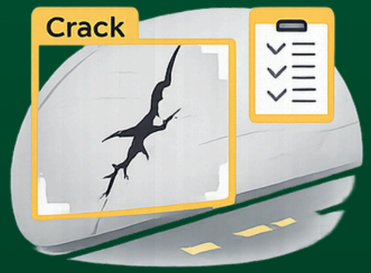
시스템 구조


시스템 아키텍처 (System Architecture)
Simulation Environment
(Gazebo Fortress + ROS2 Humble)
터널 환경
로봇 모델
LiDAR 센서
RGB-D Camera 3대
(좌 / 우 / 상단)
센서 데이터 퍼블리시
(/scan, /camera/*)
ros_gz_bridge
Gazebo ↔ ROS2
토픽 브리지
/scan
/left_camera
/right_camera
/top_camera
/cmd_vel
ROS2 Nodes
1 wall_following node
• 입력: /scan
• 기능: 좌우 거리 계산, PID 기반 중앙 유지 주행
• 출력: /cmd_vel
2 crack_detector node
• 입력: RGB image, depth image, camera_info
• 동기화: ApproximateTimeSynchronizer
• 검출: YOLOv8-crack-seg (crack bbox/mask)
• 복원: RGB-D depth를 mask 영역 3D point cloud 복원
• 후처리: mask 실패 시 bbox corner depth → center depth fallback
3 TF2 / 좌표 변환
• camera optical frame → odom
Output / Visualization
z
y
x
균열 3D 위치 복원
터널 전개도 매핑
(OpenCV)
실시간 시각화
PNG
final_tunnel_
inspection_map.png
저장
파이프라인 요약
LiDAR → wall_following → /cmd_vel → Gazebo 주행
RGB-D Camera → crack_detector → TF2 → Tunnel Unrolled Map

핵심 기능



LiDAR 기반 터널 중앙 유지 주행



YOLO 기반 균열 탐지



RGB-D Camera 기반
균열 위치 복원



Tunnel Unrolled Map 기반
점검 결과 시각화

ROS Nodes

wall_following node

- **Subscribe**: /scan
- **Publish**: /cmd_vel
- **Role**: LiDAR 기반 Wall Following으로 터널 중앙 주행 수행

ROS Nodes

crack_detector node

- **Subscribe**:
  - /left_camera, /right_camera, /top_camera
    - /image, /depth, /camera_info
  - /tf

- **Role**: YOLO 균열 탐지, depth 기반 3D 좌표 복원, 전개도 매핑 수행

- **Output**:
  - OpenCV 시각화 창
  - 최종 결과 맵 (final_tunnel_inspection_map.png) 저장

Sensor & Frame

LiDAR

- 전방, 좌측, 우측 거리 측정
- Wall Following 제어에 사용

RGB-D Camera

- left, right, top 총 3개 카메라 구성
- RGB image: 균열 탐지에 사용
- depth image: 균열 위치의 3D 복원에 사용

TF Frame

- base_link 기준 LiDAR 및 RGB-D Camera frame 구성
- 각 camera optical frame에서 odom frame으로 좌표 변환
- 변환된 3D 좌표를 터널 전개도 매핑에 사용

Implementation Method

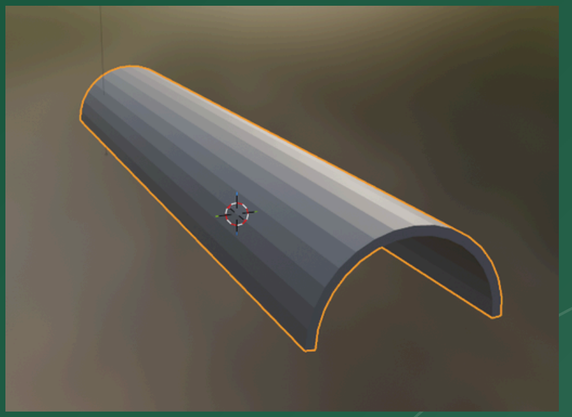
01. Project Goal
02. System Architecture
03. Implementation Method
04. Result & Demo
05. Troubleshooting & Discussion

시뮬레이션 터널 환경 구축

- Gazebo Fortress + ROS2 Humble 기반

- **터널 모델**
  1. Blender에서 반원통형 터널 모델을 제작한 뒤 OBJ로 export
  2. SDF world에서 불러와 Gazebo 환경 구성

- **균열 배치:** 검사 대상 환경 구현
  Ultralytics Crack Segmentation Dataset
  1. 균열 이미지를 터널 벽면 texture에 합성한 PNG 파일 생성
  2. OBJ/MTL 모델의 material texture로 적용
  3. 배치된 균열 plane들의 꼭짓점을 기준으로 ground-truth 생성



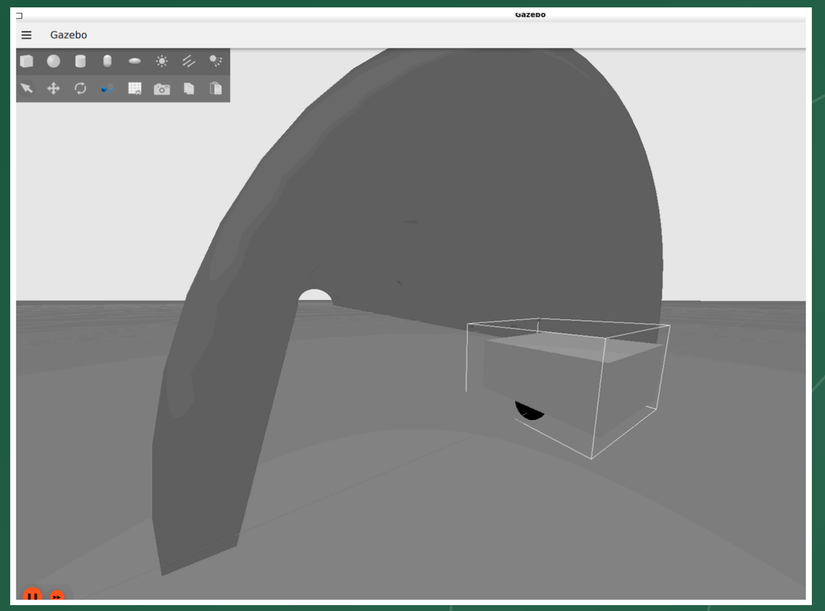
로봇 모델 및 주행 환경

로봇 구성

- TurtleBot3 Waffle 구조를 참고한 **시뮬레이션용 로봇 모델 제작**
- URDF/Xacro 기반으로 **base_link, 좌우 바퀴, 앞뒤 caster** 구성
- DiffDrive plugin을 이용해 **/cmd_vel 명령 기반 주행** 구현

주행 환경 설정

- 로봇은 **터널 입구 근처**에서 출발하도록 **초기 위치 설정**



Wall Following 기반 자율 주행

1. LiDAR **/scan** 데이터를 이용해 전방, 좌측, 우측 거리 계산

2. 좌우 거리 차이를 **중앙 유지 오차**로 정의
   - error = d_left - d_right

3. **PID 제어**를 통해 **angular velocity** 계산:
   - angular_z = kp * error + ki * error_sum + kd * error_diff

4. **linear velocity**는 **일정하게 유지**하여 터널 내부를 주행 (0.15m/s)

5. **정지 조건** 만족 시 **주행 종료**
   - 전방 거리 0.4m 미만 / 좌우 벽면 거리 합 2.0m 초과

YOLO 기반 균열 탐지

- 사전학습된 **YOLOv8 crack segmentation** 모델 사용
- 3개의 **RGB-D Camera**에서 RGB image, depth image, camera_info 수신
- ApproximateTimeSynchronizer로 RGB-D topic 시간 동기화
- RGB image에 YOLO inference를 수행하여 **균열 bounding box 및 segmentation mask 검출**

Depth 기반 3D 좌표 및 균열 크기 복원

- **Depth-based 3D Reconstruction**
  - YOLO segmentation mask 영역의 depth를 이용해 crack point cloud 복원
    - Fallback: mask depth 실패 시 bbox corners → center depth 순으로 사용
    - Outlier filtering: median/MAD 기반 depth filtering
- Camera intrinsic과 TF2를 이용해 **image coordinate → camera 3D → odom coordinate** 변환
- 3D point 분포로 **균열의 실제 길이, 폭, 대각선 길이**를 mm 단위로 추정

터널 전개도 매핑 및 위험도 시각화

- OpenCV 기반으로 반원통 터널 전개도 이미지 생성
- 터널 길이 방향 위치를 x축으로 정규화
- 터널 단면에서의 각도를 계산해 반원통 둘레 방향 위치로 변환

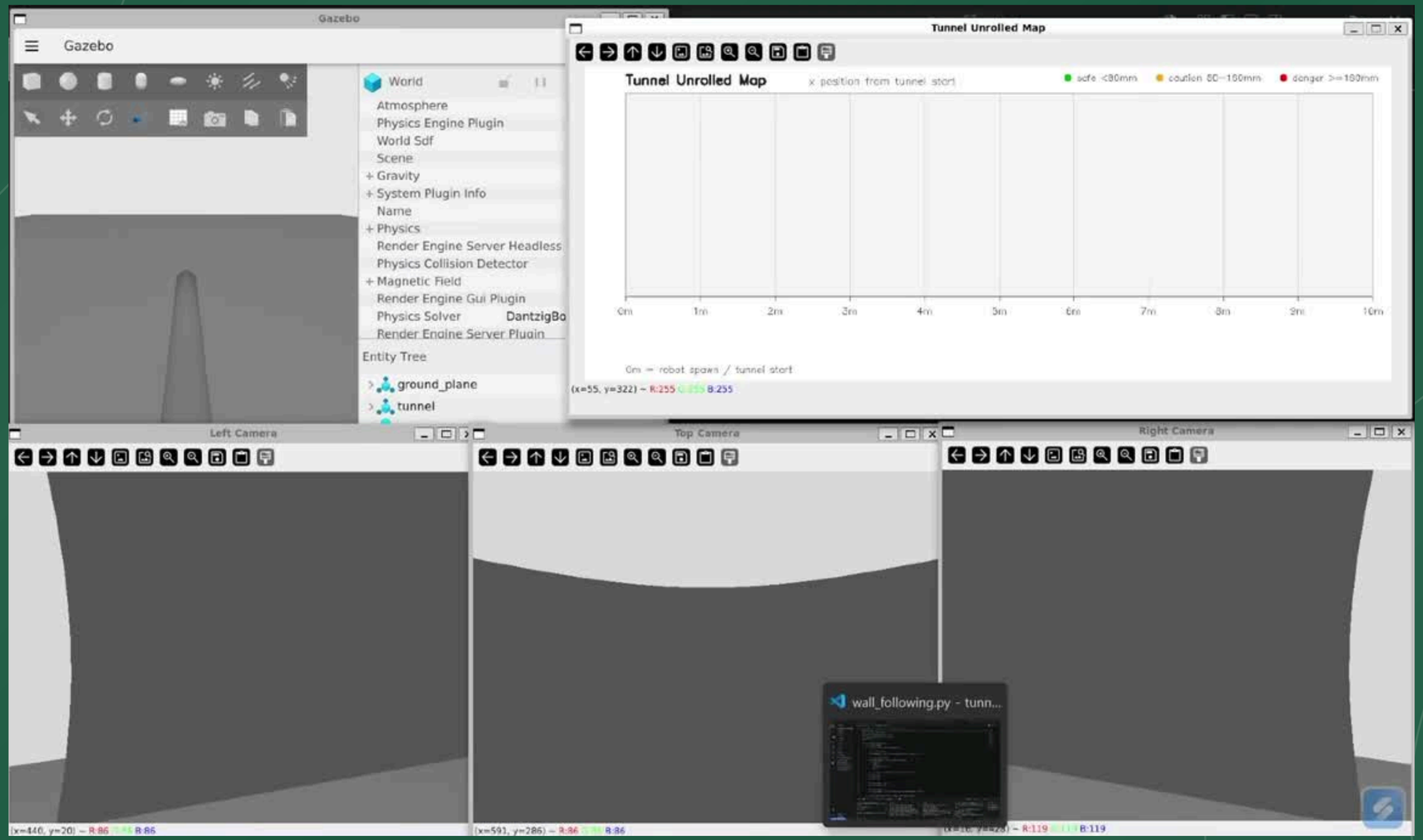
- 복원된 **균열의 추정된 3D 위치**를 전개도 위에 **표시**
  - 실제 대각선 길이를 기준으로 **시뮬레이션용 상대 위험도 등급** 시각화
  - 🟢 **안전**: 80 mm 미만, 🟠 **주의**: 80 mm 이상 160 mm 미만, 🔴 **위험**: 160 mm 이상

- 카메라 영상의 bounding box에도 동일한 위험도 색상과 실제 크기 정보를 표시하여 탐지 결과와 전개도 매핑 결과를 실시간 확인 가능

Result & Demo

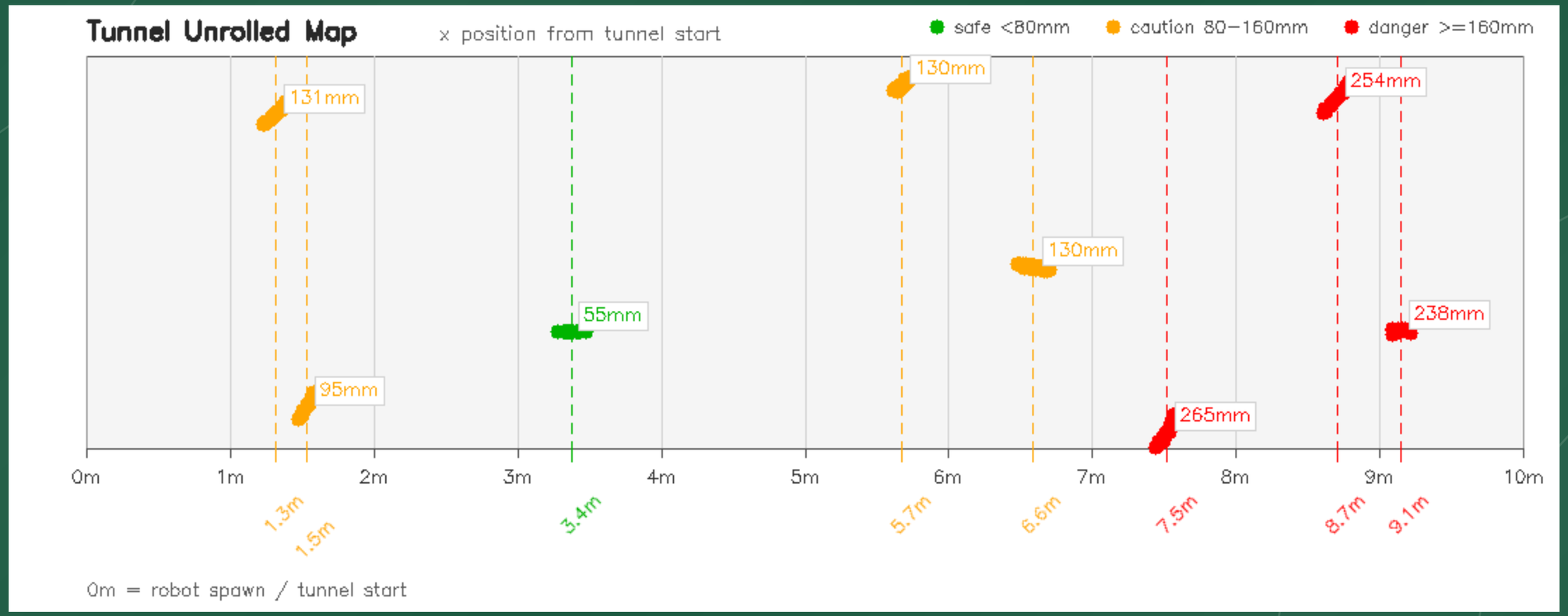
01. Project Goal
02. System Architecture
03. Implementation Method
04. Result & Demo
05. Troubleshooting & Discussion

Demo


Gazebo
Gazebo
World
Atmosphere
Physics Engine Plugin
World Sdf
Scene
Gravity
System Plugin Info
Name
Physics
Render Engine Server Headless
Physics Collision Detector
Magnetic Field
Render Engine Gui Plugin
Physics Solver
Render Engine Server Plugin
Entity Tree
ground_plane
tunnel
Tunnel Unrolled Map
Tunnel Unrolled Map
x position from tunnel start
safe <80mm
caution 80–100mm
danger >=100mm
0m
1m
2m
3m
4m
5m
6m
7m
8m
9m
10m
0m = robot spawn / tunnel start
Left Camera
Top Camera
Right Camera
wall_following.py - tunn...

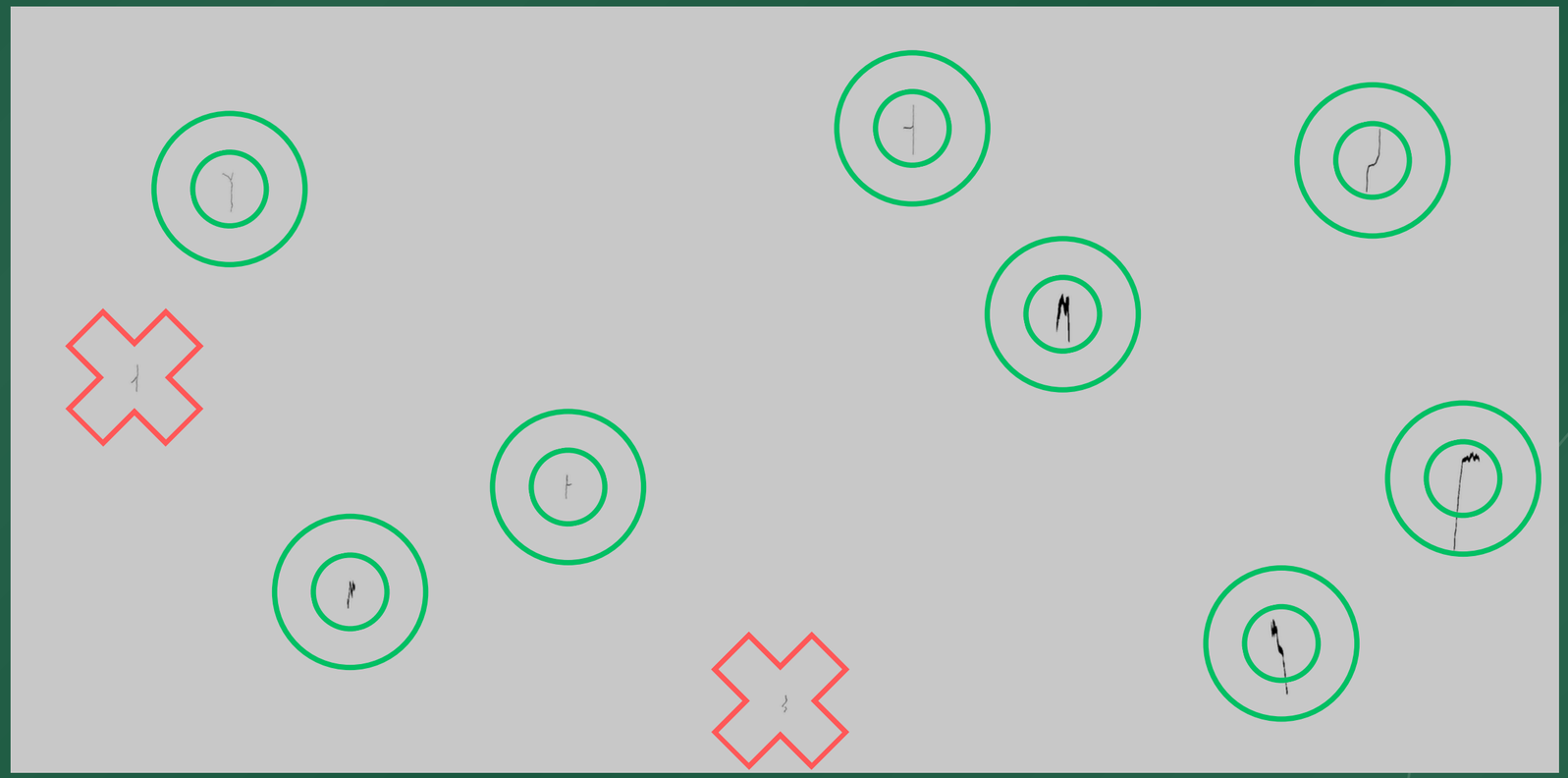
Result

터널 전개도 매핑 결과



Result

터널 전개도 매핑 결과



Analysis

- 정확도

실제 균열 수	10개
미탐지	2개
오탐지	0개
Precision	100%
Recall	80%

- 미탐지:
  - 🟢 crack_005: 45mm, 5.0m, safe
  - 🟢 crack_008: 75mm, 0.8m, safe

Analysis

- 위치 오차

MAE	0.20m
RMSE	0.24m
최대 위치 오차	0.40m
위치 상관 계수	0.997

- 크기 오차

MAE	30.75mm
RMSE	38.90mm
Bias	+2.75mm
MAPE	17.76%
최대 크기 오차	74mm
크기 상관 계수	0.857
R^2	0.707

Troubleshooting & Discussion

01. Project Goal

02. System Architecture

03. Implementation Method

04. Result & Demo

05. Troubleshooting & Discussion

Troubleshooting

Problem

- GPU 버전 호환 문제로 인해 Isaac Sim에서 Gazebo로 개발 환경을 전환

- 카메라 및 로봇 설정의 문제
  - 터널 벽면 전체를 커버하는 카메라 위치, 각도, 화각을 찾아내는 것이 어려웠음
  - 로봇 바퀴 위치 때문에 로봇이 뒤로 젖혀져서 주행해서 카메라가 의도대로 포착하지 못함
  - 로봇 스폰 위치와 로봇에 달린 카메라 위치 사이의 offset

Solution

- Gazebo 기반 시뮬레이션 환경으로 전환하여 GPU 의존도를 낮추고, ROS2 topic 및 TF 기반으로 카메라/로봇/탐지 파이프라인 구성

- 터널 반지름, 로봇 크기, 카메라 위치, 각도, 화각을 함께 고려하여 카메라별 벽면 coverage를 분석하고 사각지대 최소화
- 로봇에 caster 하나 더 추가, 카메라 장착 위치를 조정하여 주행 중 로봇 자세가 카메라 시야에 미치는 영향 완화
- 로봇 spawn 위치와 카메라 offset을 TF2 및 odom_origin_world_x 보정으로 반영하여 균열 위치를 터널 시작점 기준 좌표로 변환

Troubleshooting

Problem

- 터널 텍스처에 균열 입히는 방식에서 고전함

- 균열 탐지의 문제
  - 벽면과 지면의 경계, 터널 벽면과 터널 밖의 경계를 균열로 탐지하는 문제

Solution

- 균열 텍스처 생성 스크립트를 제작하여 터널 표면에 균열 이미지를 자동 배치하고, 균열의 실제 크기와 위치 정보를 함께 저장

- 카메라별 YOLO confidence threshold, inference image size, 카메라 각도 및 위치를 조정하여 탐지 안정성 향상
- 영상 가장자리, 과도하게 큰 bounding box, 최대 탐지 거리 초과 객체를 제외하여 터널 끝단 및 로봇 일부에 의한 오탐 감소

Discussion

- 작고 얇은 균열이 탐지되지 않는 문제

- Domain gap
  - 사전 학습 데이터는 콘크리트 등 다양한 실제 벽면의 실제 균열인 반면 우리 시뮬레이션 환경 상의 터널은 매끈한 벽면 위 평면적인 균열 텍스처

- 터널 환경이 직선 반원통 구조에 가까워 실제 터널의 복잡한 구조를 충분히 반영하지는 못함
  - 직선 터널뿐 아니라 곡선형, 폭 변화형 터널에서도 Wall Following 성능 검증 필요

- YOLO 탐지 성능은 카메라 위치, 화각, 조명 조건에 영향을 받을 수 있음
  - 실제 로봇 적용을 위해 조명 변화, 센서 노이즈, 다양한 균열 형태에 대한 robustness 개선 필요

Thank You